

Table 1: Versions and configuration options considered for each tested asset. * denotes the default setting.

Asset	Version	OCR	Layout	Tables
Docling	2.5.2	EasyOCR*	default	TableFormer (fast)*
Marker	0.3.10	Surya*	default	default
MinerU	0.9.3	auto*	doclayout_yolo	rapid_table*
Unstructured	0.16.5		hi_res with table structure	

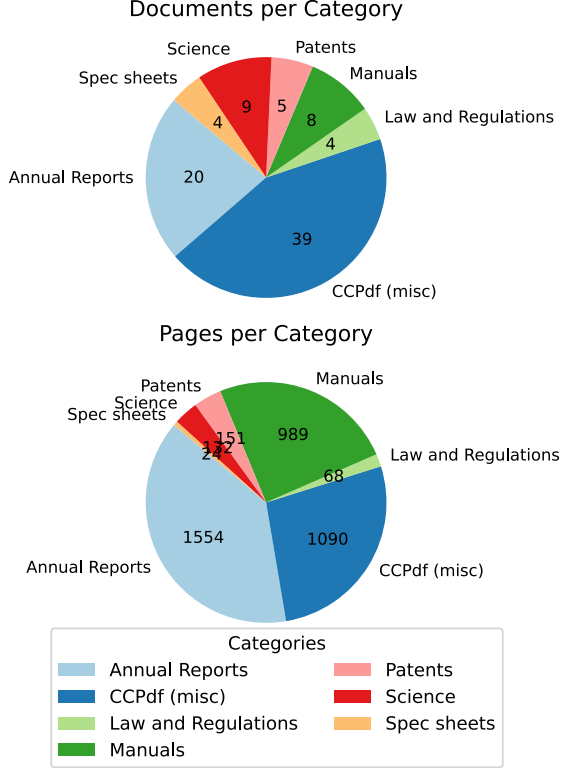


Figure 2: Dataset categories and sample counts for documents and pages.

5.3 Benchmarking Methodology

We implemented several measures to enable a fair and reproducible benchmark across all tested assets. Specifically, the experimental setup accounts for the following factors:

- All assets are installed in the latest available versions, in a clean Python environment, and configured to use the state-of-the-art processing options and models, where applicable. We selectively disabled non-essential functionalities to achieve a compatible feature-set across all compared libraries.
- When running experiments on CPU, we inform all assets of the desired CPU thread budget of 8 threads, via the `OMP_NUM_THREADS` environment variable and any accepted configuration options. The L4 GPU on our AWS EC2 VM is hidden.

- When running experiments on the L4 GPU, we enable CUDA acceleration in all accepted configuration options, ensure the GPU is visible and all required runtimes for AI inference are installed with CUDA support.

Table 1 provides an overview of the versions and configuration options we considered for each asset.

5.4 Results

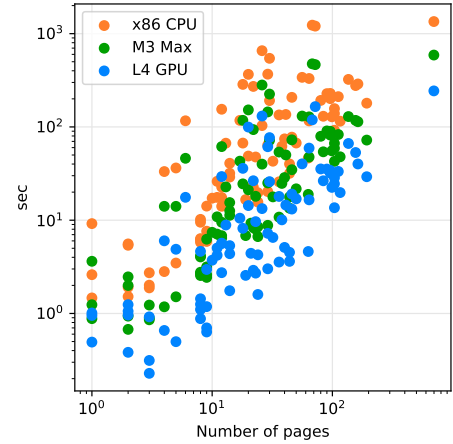


Figure 3: Distribution of conversion times for all documents, ordered by number of pages in a document, on all system configurations. Every dot represents one document. Log/log scale is used to even the spacing, since both number of pages and conversion times have long-tail distributions.

Runtime Characteristics To analyze Docling’s runtime characteristics, we begin by exploring the relationship between document length (in pages) and conversion time. As shown in Figure 3, this relationship is not strictly linear, as documents differ in their frequency of tables and bitmap elements (i.e., scanned content). This requires OCR or table structure recognition models to engage dynamically when layout analysis has detected such elements.

By breaking down the runtimes to a page level, we receive a more intuitive measure for the conversion speed (see also Figure 4). Processing a page in our benchmark dataset requires between 0.6 sec (5th percentile) and 16.3 sec (95th percentile), with a median of 0.79 sec on the x86 CPU. On the M3 Max SoC, it achieves 0.26/0.32/6.48 seconds per page (.05/median/.95), and on the Nvidia L4 GPU it achieves 57/114/2081 milliseconds per page (.05/median/.95). The

large range between 5 and 95 percentiles results from the highly different complexity of content across pages (i.e., almost empty pages vs. full-page tables).

Disabling OCR saves 60% of runtime on the x86 CPU and the M3 Max SoC, and 50% on the L4 GPU. Turning off table structure recognition saves 16% of runtime on the x86 CPU and the M3 Max SoC, and 24% on the L4 GPU. Disabling both OCR and table structure recognition saves around 75% of runtime on all system configurations.

Profiling Docling’s AI Pipeline We analyzed the contributions of Docling’s PDF backend and all AI models in the PDF pipeline to the total conversion time. The results are shown in Figure 4. On average, processing a page took 481 ms on the L4 GPU, 3.1 s on the x86 CPU and 1.26 s on the M3 Max SoC.

It is evident that applying OCR is the most expensive operation. In our benchmark dataset, OCR engages in 578 pages. On average, transcribing a page with EasyOCR took 1.6 s on the L4 GPU, 13 s on the x86 CPU and 5 s on the M3 Max SoC. The layout model spent 44 ms on the L4 GPU, 633 ms on the x86 CPU and 271 ms on the M3 Max SoC on average for each page, making it the cheapest of the AI models, while TableFormer (fast flavour) spent 400 ms on the L4 GPU, 1.74 s on the x86 CPU and 704 ms on the M3 Max SoC on average per table. Regarding the total time spent converting our benchmark dataset, TableFormer had less impact than other AI models, since tables appeared on only 28% of all pages (see Figure 4).

On the L4 GPU, we observe a speedup of 8x (OCR), 14x (Layout model) and 4.3x (Table structure) compared to the x86 CPU and a speedup of 3x (OCR), 6x (Layout model) and 1.7x (Table structure) compared to the M3 Max CPU of our MacBook Pro. This shows that there is no equal benefit for all AI models from the GPU acceleration and there might be potential for optimization.

The time spent in parsing a PDF page through our docling-parse backend is substantially lower in comparison to the AI models. On average, parsing a PDF page took 81 ms on the x86 CPU and 44 ms on the M3 Max SoC (there is no GPU support).

Comparison to Other Tools We compare the average times to convert a page between Docling, Marker, MinerU, and Unstructured on the system configurations outlined in section 5.2. Results are shown in Figure 5.

Without GPU support, Docling leads with 3.1 sec/page (x86 CPU) and 1.27 sec/page (M3 Max SoC), followed closely by MinerU (3.3 sec/page on x86 CPU) and Unstructured (4.2 sec/page on x86 CPU, 2.7 sec/page on M3 Max SoC), while Marker needs over 16 sec/page (x86 CPU) and 4.2 sec/page (M3 Mac SoC). MinerU, despite several efforts to configure its environment, did not finish any run on our MacBook Pro M3 Max. With CUDA acceleration on the Nvidia L4 GPU, the picture changes and MinerU takes the lead over the contenders with 0.21 sec/page, compared to 0.49 sec/page with Docling and 0.86 sec/page with Marker. Unstructured does not profit from GPU acceleration.

6 Applications

Docling’s document extraction capabilities make it naturally suitable for workflows like generative AI applications (e.g., RAG), data preparation for foundation model training, and fine-tuning, as well as information extraction.

As far as RAG is concerned, users can leverage existing Docling extensions for popular frameworks like LlamaIndex and then harness framework capabilities for RAG components like embedding models, vector stores, etc. These Docling extensions typically provide two modes of operation: one using a lossy export, e.g., to Markdown, and one using lossless serialization via JSON. The former provides a simple starting point, upon which any text-based chunking method may be applied (e.g., also drawing from the framework library), while the latter, which uses a swappable Docling chunker type, can be the more powerful one, as it can provide document-native RAG grounding via rich meta-data such as the page number and the bounding box of the supporting context. For usage outside of these frameworks, users can still employ Docling chunkers to accelerate and simplify the development of their custom pipelines. Besides strict RAG pipelines for Q&A, Docling can naturally be utilized in the context of broader agentic workflows for which it can provide document-based knowledge for agents to decide and act on.

Moreover, Docling-enabled pipelines can generate ground truth data out of documents. Such domain-specific knowledge can make significant impact when infused to foundation model training and fine-tuning.

Last but not least, Docling can be used as a backbone for information extraction tasks. Users who seek to create structured representations out of unstructured documents can leverage Docling, which maps various document formats to the unified `DoclingDocument` format, as well as its strong table understanding capabilities that can help better analyze semi-structured document parts.

7 Ecosystem

Docling is quickly evolving into a mainstream package for document conversion. The support for PDF, MS Office formats, Images, HTML, and more makes it a universal choice for downstream applications. Users appreciate the intuitiveness of the library, the high-quality, richly structured conversion output, as well as the permissive MIT license, and the possibility of running entirely locally on commodity hardware.

Among the integrations created by the Docling team and the growing community, a few are worth mentioning as depicted in Figure 6. For popular generative AI application patterns, we provide native integration within LangChain (Chase 2022) and LlamaIndex (Liu 2022) for reading documents and chunking. Processing and transforming documents at scale for building large-scale multi-modal training datasets are enabled by the integration in the open IBM data-prep-kit (Wood et al. 2024). Agentic workloads can leverage the integration with the Bee framework (IBM Research 2024). For the fine-tuning of language models, Docling is integrated in InstructLab (Sudalairaj et al. 2024),